

LEYAN LI

leyanpower@gmail.com | +44 7887 930137 | GitHub: github.com/lionlepower | leyan-li.com

Edinburgh, UK · open to relocation (UK / EU) · Right to work in the UK: Student visa (full-time work permitted post-course); applying for the 2-year Graduate Route — no sponsorship required.

PROFILE

MSc HPC & Data Science graduate targeting AI / agent engineering roles. Built and evaluated a tool-calling LLM agent over real supercomputer data — deterministic tools, rule baseline, 30-question eval set, prompt ablation, 229 offline tests — on top of 4,096-core performance work on ARCHER2.

EDUCATION

University of Edinburgh — MSc High Performance Computing with Data Science | Sep 2025 – Dec 2026 (expected). Taught courses and dissertation completed Aug 2026 — **available immediately**; degree conferred Dec 2026

University of Liverpool / XJTLU — BSc Computer Science (2+2), **First-Class Honours** | Sep 2021 – Jun 2025

TECHNICAL SKILLS

Languages: C / C++, Python, Bash, SQL

LLM & agents: OpenAI-compatible function calling / tool-schema design, system-prompt policy, offline evaluation sets and ablations, structured tracing; working knowledge of RAG, planning / memory / reflection patterns and LangGraph-style orchestration; PyTorch, NumPy, pandas

Backend & services: FastAPI + Pydantic, SQLite / MySQL (indexing, transactions, query tuning), REST / HTTP, JSON Lines structured logging, exception → HTTP status mapping; working knowledge of rate limiting, idempotency, Redis / TTL, Docker

Engineering: Linux, Git, pytest, GitHub Actions CI; reproducible pipelines with regression tests that pin key figures

Parallel & performance: MPI, OpenMP, PETSc, Slurm, Linaro MAP; strong / weak scaling, NUMA and thread affinity

PROJECTS

Evaluable Tool-Calling LLM Agent over Real Supercomputer Benchmark Data

| Python · SQLite · FastAPI · Pydantic · OpenAI SDK (function calling) · pytest lionlepower/HPC_Agent

Structured real PETSc benchmark results from ARCHER2 into SQLite and wrapped each analysis task as a deterministic tool: the LLM only decides which tool to call and with what arguments; every number comes from deterministic code, so agent correctness becomes measurable.

- **Layered architecture with an isomorphic contract:** Six layers (ingestion / metrics / tools / retrieval / routing / observability) behind two peer process boundaries, CLI and FastAPI. Rule router and LLM tool call emit the same shape (intent + arguments), so one evaluation set scores both and switching is a one-line dispatch change. Contracts pinned by 229 automated tests that run fully offline with no API key.
- **Function calling and defensive parsing:** Tool-schema descriptions state each tool's boundary; the size argument is deliberately optional — omitting it means “ask back”, calling no tool means “decline”. No tool call, hallucinated tool name and malformed JSON all collapse to an explicit unknown: the system never guesses an argument, so never returns a silently wrong number. API failures propagate rather than falling back to rules, keeping the evaluation clean.
- **Evaluation and ablation:** Self-built 30-question set in five categories (normal / ask-back / decline / number trap / adversarial); the expected value is the tool call, not answer text, so the harness is router-agnostic. Rules 83.3%, LLM 100%; the same model without its system prompt drops to 63.3% — accuracy is driven by tool documentation and policy prompt, not the model. Limits stated (30 questions, one model, one run).
- **Observability, API and minimal RAG:** trace_id-threaded JSON Lines event stream replays any execution; redaction centralised in one function and enforced by a scanning test. FastAPI POST /ask: invalid router → 422 via Pydantic, missing key → 503 not 500. Minimal RAG with IDF-weighted lexical retrieval (standard library, deliberately no vector store); no hit → nothing injected, and with RAG off the messages are byte-identical to the evaluation, pinned by a test.

PETSc Layout Study at 4,096 Cores on ARCHER2 — the agent's data source and credibility baseline

| C · PETSc · MPI · OpenMP · Slurm · Linaro MAP · Python / pandas · CI lionlepower/HPC_PETSC_Benchmark

A controlled experiment on ARCHER2 (dual-socket AMD EPYC 7742, 128 cores/node): with total cores fixed, how to split MPI ranks vs. OpenMP threads — values usually left to job-script defaults and rarely measured. 2D/3D stencil + CG+GAMG, 5M–165M unknowns, four full-node layouts, up to 32 nodes.

- **Scope and a metric of my own:** Pushed to 32 nodes / 4,096 cores, where saturation becomes visible. Linaro MAP perturbs runtime, so it only ranks layouts by share; PETSc -log_view has zero overhead and covers all 264 weak-scaling runs, on which I defined an additive “synchronisation exposure share” — nested leaf events sum to 116–372% of KSPSolve, but halo exchange and global reductions sit on disjoint call paths, so their sum is legitimate. At 32 nodes it falls from 48.2% (flat MPI) to 26.4% (16×8), making “why is this layout faster” a per-point measurement.
- **Results (summary):** Weak scaling: best 2D layout 64×2 (794.8M equations/s at 165M / 32 nodes, +13.1% over flat MPI), best 3D 32×4 (up to +33.2%); fixed-20M strong scaling: 2D 29.83× at 93.2% cumulative efficiency, 3D 16×8 3.81× faster than flat MPI. The optimum shifts with dimension, size and per-core work; MAP locates the mechanism as the crossing of MPI exposure (46.7% → 6.2%) and OpenMP wait (48.3% → 76.5%).

Value Factorisation in Multi-Agent Actor-Critic Methods (BSc dissertation)

| Python · PyTorch · PyMARL · MAPPO / VDN / QMIX · PettingZoo lionlepower/MARL_VALUE_FACTORISATION

- **Counter-intuitive result and root cause:** Single-variable comparison of VDN additive and QMIX monotonic critics inside MAPPO (shared runner, buffer, actor, PPO hyperparameters). The more expressive critic did not win: on sparse-reward Predator-Prey (300k steps) only the additive critic reached positive return (≈150); QMIX's critic loss opened at 5.4×10^4 and diverged again. Root cause: the inherited value-based target $\max_a Q_i$ against an on-policy actor makes td_error systematically optimistic.

Publication: Li, Leyan. Convolutional Neural Networks Based Medical Image Analysis. EMITI 2024.